

# Assignment 4: Backend Book Management API

---

## Introduction

This assignment is based on:

- **Week 6:** HTTP and REST Fundamentals (request/response cycle, HTTP methods, route parameters)
- **Week 7:** Express Backend Development (server setup, routing, JSON handling)
- **Week 8:** SQL Fundamentals and Persistence (tables, primary keys, SELECT, INSERT, UPDATE, DELETE)

This assignment builds directly on the concepts and lab work completed in Weeks 6–8.

You are required to develop a backend API that integrates an Express server with a SQLite database.

No frontend interface is required.

---

## Learning Outcomes

After completing this assignment, you should be able to:

- Explain how HTTP requests are handled by an Express server
  - Implement REST-style API endpoints
  - Connect a backend server to a relational database
  - Perform basic database operations using SQL
  - Demonstrate data persistence across server restarts
  - Implement full CRUD functionality (Create, Read, Update, Delete)
- 

## Assignment Description

You will implement a **Book Management API** that allows clients to:

- Retrieve all books
- Retrieve a single book by ID
- Add a new book
- Update an existing book
- Delete an existing book

All book data must be stored persistently using **SQLite**.

---

## Task 1: Project Setup (15%)

1. Create a new project directory.
  2. Initialise the project using: `npm init -y`
  3. Install the required dependencies: `npm install express sqlite3`
  4. Create a file named `server.js`.
  5. Start an Express server listening on port 3000.
-

## Task 2: Database Setup (20%)

1. Create a SQLite database file named **database.db**.
  2. Create a table named **books** with the following columns:
    - id (INTEGER, primary key, auto-increment)
    - title (TEXT, required)
    - author (TEXT, required)
    - year (INTEGER)
    - status (TEXT, required)
  3. The **status** field must only allow the following values:
    - to-read
    - reading
    - completed
  4. Use **CREATE TABLE IF NOT EXISTS** to avoid duplicate table creation.
- 

## Task 3: Read Operations (25%)

### 3.1 GET /books

- Implement a GET endpoint that returns all books as JSON.

### 3.2 GET /books/:id

- Implement a GET endpoint that retrieves a book by ID.
- Use a **WHERE clause** to filter results.
- Return an appropriate response if the book does not exist (e.g., **404 Not Found**).

### 3.3 GET /books?status=reading

- Extend the GET endpoint to support filtering using a query parameter.
  - If a status value is provided, return only books matching that status.
- 

## Task 4: Create, Update and Delete Operations (30%)

### 4.1 POST /books

- Implement a POST endpoint to add a new book.
- Read **title, author, year, and status** from the request body.
- Validate that **title, author, and status** are provided.
- Validate that status is one of the allowed values.
- Use a **parameterised SQL query**.
- Return the **ID of the newly created book**.

### 4.2 PUT /books/:id

- Implement a PUT endpoint to update an existing book.
- Allow updating **title, year, and status only**.
- Validate status if it is included in the request.
- Use a **WHERE clause** to target the correct record.
- Return an appropriate response if the book does not exist.

#### 4.3 DELETE /books/:id

- Implement a DELETE endpoint to remove an existing book.
- Use a **WHERE clause** to delete the correct record.
- Return a suitable success message if deletion is successful.
- Return an appropriate response (e.g., **404 Not Found**) if the book does not exist.

---

### Task 5: Persistence Verification (10%)

1. Insert at least **two books** into the database.
2. Restart the server.
3. Verify that the data is still stored and can be retrieved.

---

### Submission Requirements

Submit a **ZIP file** containing:

- `server.js`
- `database.db`
- `package.json`
- `package-lock.json`
- Screenshots demonstrating:
  - Server running
  - `GET /books`
  - `GET /books/:id`
  - `GET /books?status=reading`
  - `POST /books`
  - `PUT /books/:id`
  - `DELETE /books/:id`

---

### Notes

- Do not implement a frontend.
- Do not use external frameworks beyond Express.
- Do not use cloud-based databases.
- All API responses must be returned in **JSON format**.
- Use appropriate **HTTP status codes** for success and error responses.